

Agent Fabric

Building production-grade LLM agents

LangChain · LangGraph · RAG · MCP · FastAPI · SQL Agent

Trainers Nguyễn Hữu Nhật Thành · Nguyễn Ngọc Quý

June 2026 · via Zoom

SCHEDULE

Course at a glance

SESSION 1

Foundations of Agent Building

LangChain · LangGraph · RAG



Thursday, 4 June 2026

Online via Zoom

What you will build



Objective 1 · Module 1 · LLM applications with LangChain

Chains, prompt templates, and tool-calling agents that compose into real workflows.



Objective 2 · Module 2 · Stateful multi-agent flows with LangGraph

State management, conditional branching, and checkpointing for resilient pipelines.



Objective 3 · Module 3 · RAG over your own data

End-to-end retrieval-augmented generation with vector stores and document retrievers.

How this course is structured



Module 1 · LangChain — LLM applications

Session 1 · Chains, prompt templates, and tool-calling agents.



Module 2 · LangGraph — stateful multi-agent flows

Session 1 · State, nodes, edges, conditional branching, and checkpointing.



Module 3 · RAG — retrieval-augmented generation

Session 1 · Ingest, retrieve, generate — vector stores and document retrievers.

PREREQUISITES & APPROACH

Before we start

PREREQUISITES



Python — comfortable with OOP, async/await, asyncio.



LLM basics — prompting & API calls (OpenAI / Anthropic / Gemini / Ollama).



Data — solid grasp of JSON and REST APIs.



FastAPI — routes, request/response models, run a server.

METHODOLOGY

30%

Theory & concepts

70%

Hands-on coding



Have your IDE & API keys ready before each session.

SESSION 1 · 4 JUNE 2026

Foundations

LangChain → LangGraph → Retrieval-Augmented Generation

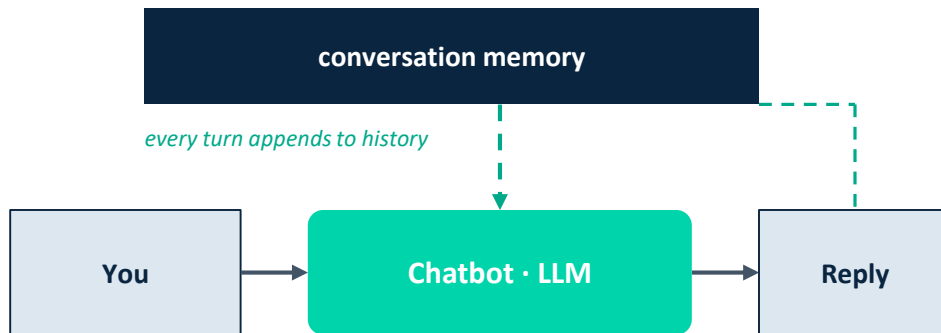
What is a chatbot?



THE IDEA

A program you talk to in plain language.

An LLM writes each reply. The whole conversation so far is fed back in every turn — so it remembers what was already said and can follow up naturally.



TWO MOVING PARTS

- ▶ The model — turns your text into a reply
- ▶ The memory — the running conversation history

You: “Who wrote Dune?” **Bot:** “Frank Herbert.”

You: “When?” ← *only makes sense because it remembers*

A chatbot = a language model + memory of the conversation. Remove the memory and every message starts from zero.

What is an agent?



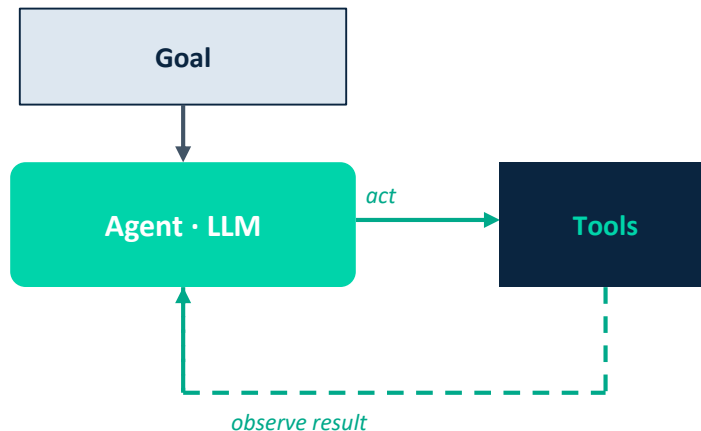
THE IDEA

A chatbot that can act, not just talk.

Give it a goal. It reasons about what's needed, reaches for a tool — search, a calculator, an API, a database — runs it, reads the result, and repeats until it can answer.

THE LOOP

Think → Act → Observe → repeat



a tool is anything the model can call

A chatbot answers from what it already knows. An agent takes actions to go find out.

What is a RAG chatbot?

RAG = Retrieval-Augmented Generation. A chatbot that looks things up in your documents before it answers.

WHY

A plain LLM doesn't know your private or recent data, and may confidently make things up. RAG feeds it the real passages so answers stay grounded — and can cite their source.

PREPARE · once, ahead of time



ANSWER · each question



What is an agentic application?



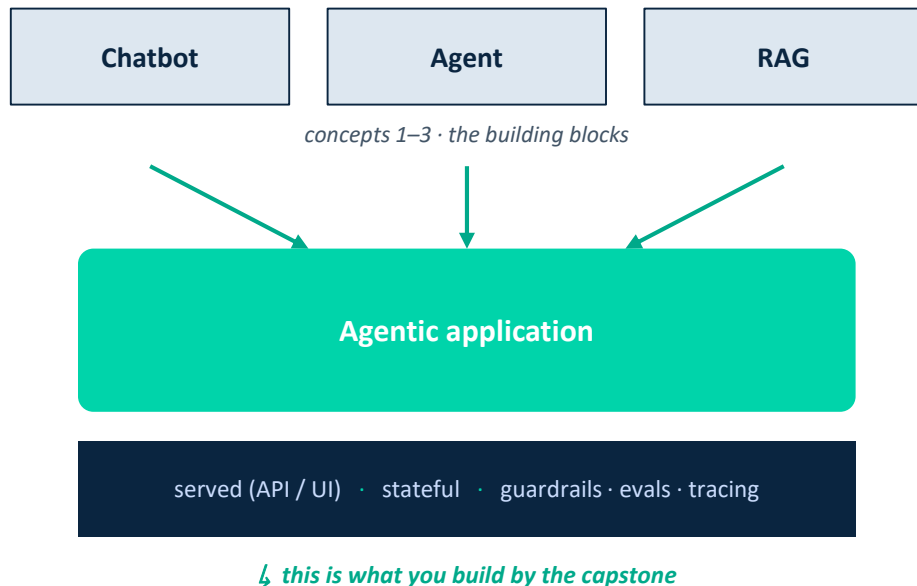
THE IDEA

The whole system — not just one agent.

Chatbot, agent, and RAG are building blocks. An agentic application wires them into a real product — served behind an API or UI, holding state, with guardrails and tracing around it.

WHAT IT ADDS

- ▶ Orchestration — routing & multi-step flows
- ▶ A serving layer & guardrails for real users



The patterns are pieces. An agentic application is the assembled, served, observable product.

LangChain: the LLM toolkit

The universal glue layer for building LLM-powered applications.

WHY

The problem it solves

Raw LLM calls lack memory, tool access, streaming, and composability. LangChain wraps them so you stop reinventing scaffolding.

WHAT

Key primitives

Models, Prompts, Chains, Tools, Agents — LEGO bricks for LLM apps. Small, swappable, composable.

BUILD

What you will build

A tool-calling agent that decides whether to answer from memory or reach for external tools per user query.

COVERS

Slides 12 – 14

LangChain overview, LCEL & Runnables, from chains to tool-calling agents.

LangChain: the LLM toolkit

Why it exists

An LLM call alone isn't an application. You need prompts, memory, tool access, streaming, retries, and a way to compose them. LangChain provides those primitives so you can stop reinventing scaffolding.

Think of it as

LEGO bricks for LLM apps — small, swappable, composable.



Models

Uniform API across providers



Prompts

Templates with variables & few-shots



Chains

Sequence steps into pipelines



Tools & Agents

Let the LLM choose what to call

A LangChain pipeline in code

agent.py

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = PromptTemplate.from_template(
    "Summarize for a developer: {text}"
)

chain = prompt | llm
result = chain.invoke({"text": doc})
```

The LCEL pipe

Components are composed with the `|` operator, just like a Unix pipe. Each piece is a Runnable — testable in isolation, swappable at any point.



invoke · stream · batch · async — all for free

From chains to tool-calling agents

An agent is just an LLM that decides — at each step — whether to answer or call a tool. The runtime loops until the model says "done".

1

User asks

"What's the weather in HCMC?"

2

LLM decides

Needs a tool — emits a function call

3

Tool runs

Your code calls the weather API

4

LLM answers

Reads the result, replies in plain text

🔄 Steps 2–3 repeat until the model returns a final answer

LangGraph: stateful multi-agent flows

When chains aren't enough — add cycles, state, and coordination.

WHY

The problem it solves

Chains are linear. Real workflows need cycles, retries, branching, and the ability to pause and resume mid-run.

WHAT

Core concepts

State (typed dict), Nodes (functions), Edges (routing logic) — plus checkpoints for pause/resume.

BUILD

What you will build

A multi-agent graph with conditional branching: agent node, tool node, loop-back edge until done.

COVERS

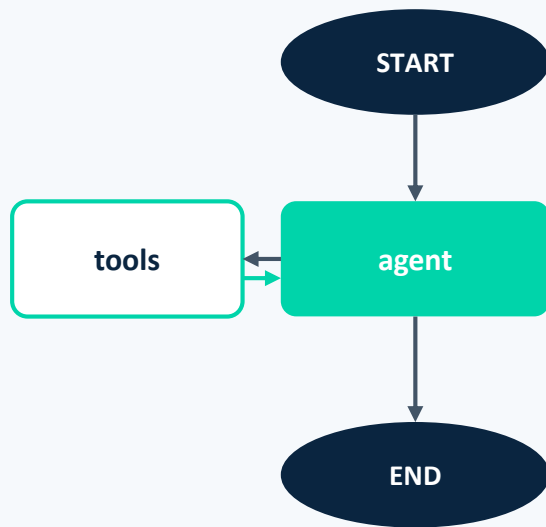
Slides 16 – 24

Workflows vs agents, LLM augmentations, chaining, routing, orchestrator-worker, evaluator-optimizer.

When chains aren't enough

Chains break when...

- ✓ You need cycles — retry, refine, self-critique
- ✓ Multiple agents must coordinate or hand off
- ✓ Flow depends on intermediate state
- ✓ You need to pause, persist, and resume runs



nodes do work · edges decide what's next

Three concepts you need



State

A typed dict that flows through the graph. Each node reads from it and returns updates — reducers merge them in.



Nodes

Plain functions or Runnables that take state and return a partial state update. This is where the work happens.



Edges

Connect nodes. Conditional edges branch based on state — that's how you get cycles and routing.

Bonus: *checkpointers persist state between runs — your agent can pause, hand off to a human, and resume hours later.*

Workflows and agents

Workflows run predetermined code paths in a fixed order. Agents are dynamic — they define their own process and tool usage at runtime. Both patterns are supported in LangGraph with persistence, streaming, and built-in deployment.

WORKFLOWS

Predetermined paths

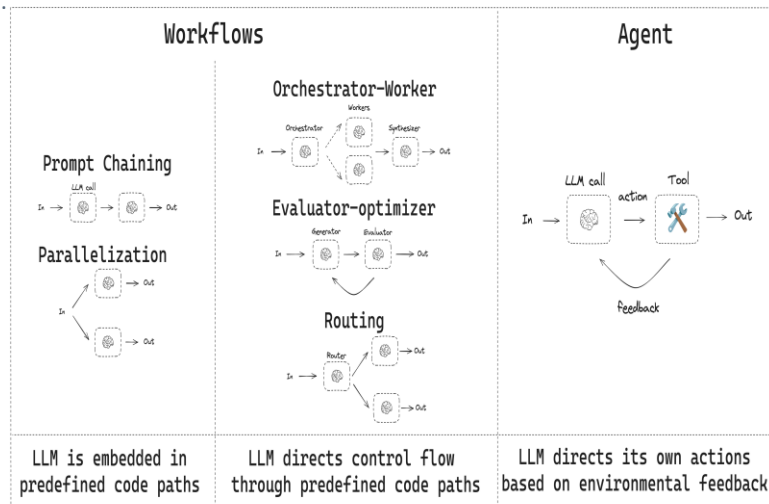
Each step is designed ahead of time. Best for well-defined, repeatable tasks — document translation, data validation, multi-step pipelines.

LangGraph advantage: persistence, streaming, debugging, and deployment support for both patterns.

AGENTS

Dynamic, self-directed

The LLM decides which tools to call and how. Best for open-ended problems where the solution path can't be predefined.



LLMs and augmentations

Workflows and agents are built on LLMs with augmentations that tailor their behaviour.

TOOL CALLING

Let the LLM act

Bind functions as tools. The model emits `tool_calls`; your code executes them and returns `ToolMessage` results for the next turn.

Composable: use tool calling + structured output + memory together in the same LLM call.

STRUCTURED OUTPUT

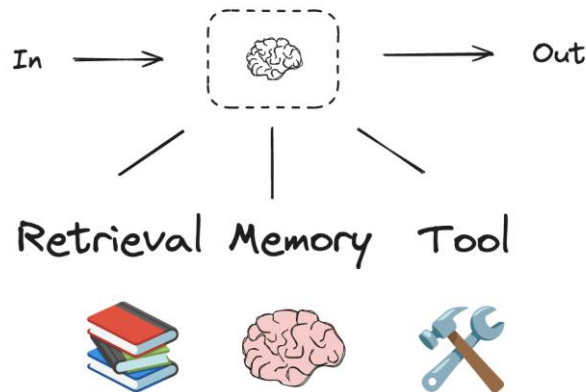
Force a schema

`.with_structured_output(Schema)` makes the LLM return a typed Pydantic object — ideal for routing decisions, planning, and evaluation grades.

SHORT-TERM MEMORY

Track context

Pass the full message history each invocation. The LLM uses prior turns for multi-turn dialogue and self-correction across steps.



Prompt chaining

Each LLM call processes the output of the previous one — ideal for well-defined tasks that break into smaller, verifiable steps.

01 Generate

First LLM call produces a draft. e.g. write a joke about a given topic.

02 Gate

A conditional edge checks quality. Fail → improve; Pass → next step.

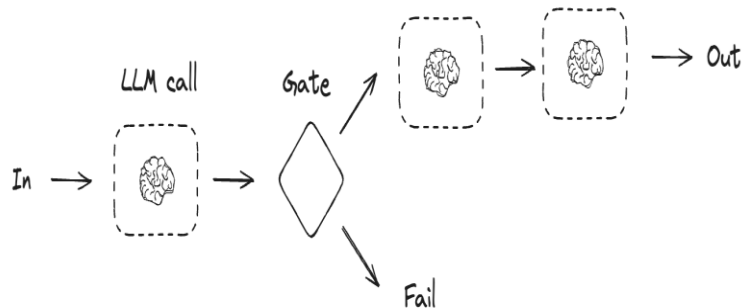
03 Improve

Second LLM call refines the output, e.g. adds wordplay or fixes tone.

04 Polish

Third LLM call adds a final touch before the result exits the graph.

Key API: `add_conditional_edges()` lets you skip or loop steps based on intermediate state.



Parallelization

Multiple LLM calls run simultaneously — splitting a task into parallel subtasks, or running the same task multiple times to increase confidence.

SPLIT SUBTASKS

Speed via parallel fan-out

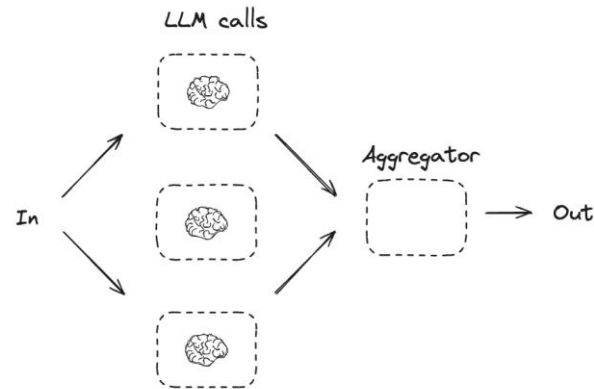
Fan out from START to N independent worker nodes — e.g. joke + story + poem — then an aggregator node merges all outputs.

In code: `add_edge(START, node_a) + add_edge(START, node_b)` fans out; all edges to aggregator fan back in.

MULTIPLE RUNS

Confidence via repetition

Send the same input to N nodes with different criteria — e.g. score for citation quality, source diversity, and accuracy simultaneously.



Routing

A router node reads the input and directs it to the most appropriate specialised handler.
Use this when different inputs need completely different processing.

CLASSIFY

Router node

An LLM with structured output reads the input and emits a typed decision — e.g. {step: 'joke'}.

Key: `llm.with_structured_output(Route)` produces a typed decision — no string parsing needed.

DISPATCH

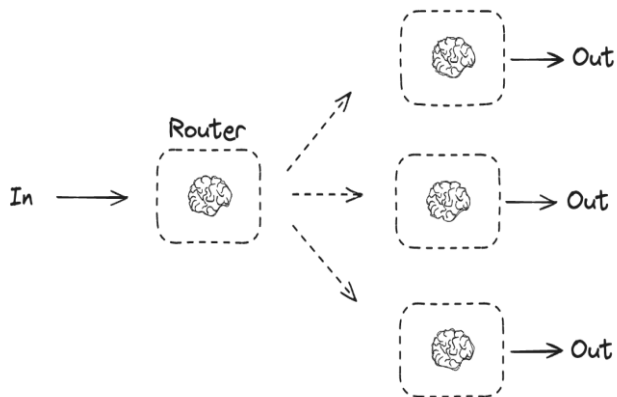
Conditional edge

`add_conditional_edges()` maps each decision value to the correct target node name.

SPECIALIZE

Dedicated handlers

Each downstream node has its own prompt, tools, and output format — purpose-built for its task.



Orchestrator-worker

An orchestrator LLM plans a complex task into subtasks and dynamically spawns worker nodes. Unlike parallelization, subtasks are determined at runtime — more flexible for open-ended work.

ORCHESTRATE

Plan the work

Orchestrator LLM calls `.with_structured_output(Sections)` to produce a typed list of subtasks.

vs. Parallelization: fixed branches vs. runtime-determined branches — use orchestrator-worker for unknown number of subtasks.

DISPATCH

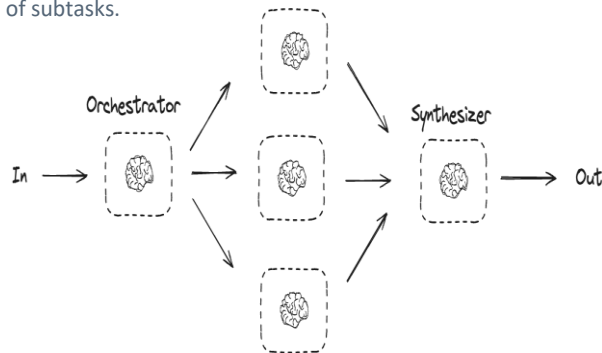
Send() API

LangGraph's `Send()` API creates one worker node per subtask dynamically — each with its own isolated `WorkerState`.

SYNTHESIZE

Merge all outputs

Worker outputs accumulate in a shared state key via `operator.add reducer`. A synthesizer node joins them into the final result.



Evaluator-optimizer

One LLM generates; a second evaluates against success criteria. If rejected, feedback routes back to the generator — looping until accepted.

01 GENERATE

LLM produces an initial response — e.g. a joke, translation, or code solution.

02 EVALUATE

Evaluator LLM grades with structured output: {grade: 'funny'|'not funny', feedback: '...'}

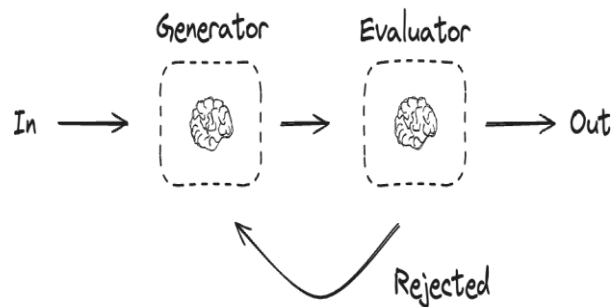
03 ROUTE

Conditional edge: Accepted → END; Rejected → back to generator with feedback.

04 REFINE

Generator re-runs with evaluator feedback appended to the prompt, producing an improved version.

Best for: tasks with clear quality criteria but no single right answer — translation fidelity, tone, code correctness.



Agents

Agents are LLMs in continuous feedback loops that decide which tools to call. Unlike workflows, the execution path is not predetermined — the LLM drives it.

llm_call NODE

Think

Receives full message history + system prompt.
Decides: answer directly, or emit a tool_call?
Routed to tool_node if tool_calls is non-empty.

Shortcut: from langgraph.prebuilt import create_react_agent — gives you this full loop out of the box.

tool_node NODE

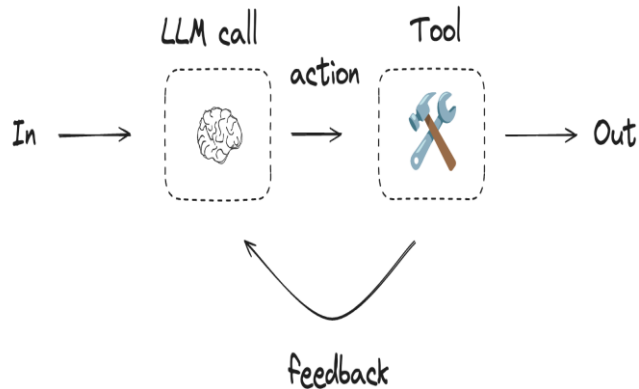
Act

Executes each tool_call, collects results as ToolMessages, and appends them to the message list for the next LLM turn.

should_continue

Loop or stop

Conditional edge: tool_calls present → tool_node.
No tool_calls → END. Loop continues until the LLM produces a final text answer.



Retrieval-Augmented Generation

Ground your LLM in your own documents — no hallucinations, with citations.

WHY

The problem it solves

LLMs don't know your private data or recent events and will confidently invent answers. RAG feeds them the real source passages.

WHAT

Three-step pipeline

Ingest (chunk + embed docs)
→ Retrieve (top-K similarity search) → Generate (LLM reads context, answers).

BUILD

What you will build

A Chroma-backed retriever tool wired into a LangGraph agent that chooses docs vs web search per query.

COVERS

Slides 27 – 28

RAG concepts, full pipeline architecture, vector store selection (Chroma / FAISS / pgvector / Pinecone).

Retrieval-Augmented Generation

LLMs know what they were trained on — not your docs, your tickets, or last week's contract. RAG bridges that gap.

1

INGEST

Chunk docs, embed them, store vectors.

2

RETRIEVE

Embed the query, fetch the top-K nearest chunks.

3

GENERATE

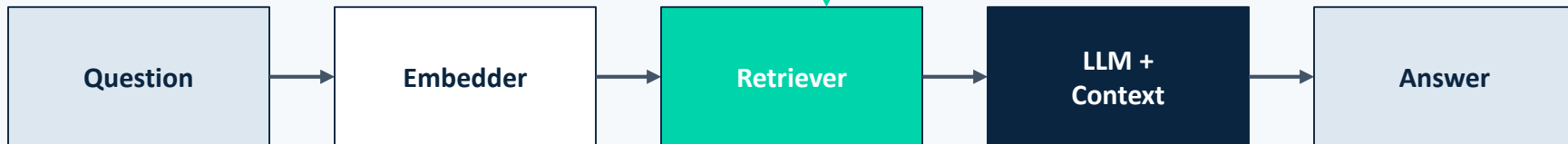
Stuff chunks into the prompt as grounded context.

The RAG pipeline, drawn out

INGEST · offline, batch



QUERY-TIME · each user request



similarity search

Same embedding model on both sides — that's what makes the similarity meaningful.

Picking the right vector store

Tool	Best for	Trade-off
Chroma	Local dev, small prototypes	Not for production scale
FAISS	In-process, very fast, single-node	No managed persistence layer
pgvector	You already run Postgres	Tune HNSW params for recall
Pinecone	Managed, multi-tenant SaaS	Cost grows with index size
Qdrant / Weaviate	Self-host with filters & metadata	Operational overhead

Start with Chroma. Switch when scale or latency forces you to.

Hands-on: your first agent + RAG

GOAL A research assistant that searches your docs (RAG) and the web — picking the right tool per question, orchestrated with LangGraph.

EXPECTED OUTCOME Agent answers doc questions with chunk citations and web questions with live sources. Console shows routing decisions (RAG vs web) per query. Graph state visible in Langfuse trace.

1 Set up

Fresh venv, then pip install langchain langgraph chromadb. Export your API key.

2 Ingest docs

Load the sample docs, split into chunks, embed them into a Chroma index.

3 Retriever tool

Wrap the vector store as a tool the agent can call to fetch relevant chunks.

4 Web-search tool

Add a second tool, so the agent has a real choice of where to look.

5 Build the graph

Wire agent + ToolNode + tools_condition, with a loop back to the agent.

6 Run & probe

Ask questions; watch it route between your docs and the web per query.



Thank you.

Questions, ideas, sharp critiques — all welcome.

REACH OUT

Trainers: **Nguyễn Hữu Nhựt Thành** · **Nguyễn Ngọc Quý**

Registration: training@tma.com.vn

SEE YOU ON ZOOM

4 June 2026

14:00 – 16:00 ICT